

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..21

ADMIN OPERATIONS

 Copy as Markdown

Firestore admin operations

The Firestore identity provider provides admin operations through `FirestoreIdpAdmin` for managing Firestore-authenticated accounts on the server. Common use cases include linking an existing Serverpod user to a Firestore account, looking up accounts for support tools, and cleaning up orphaned accounts.

WARNING

Admin operations should only be called from secure server-side code. Do not expose these methods directly through client endpoints without proper authorization checks.

Accessing the FirestoreIdpAdmin

You can access the admin operations through the `AuthServices.instance.firestoreIdp` property. This requires that the Firestore identity provider is already configured (see [setup](#)).

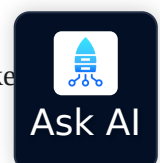
```
import 'package:serverpod_auth_idp_server/providers/firebase.dart';
import 'package:serverpod_auth_idp_server/core.dart';

final firestoreIdp = AuthServices.instance.firestoreIdp;
final admin = firestoreIdp.admin;
```

Finding accounts

Pick the finder that matches the identifier you have on hand:

- `findAccountByEmail` when you have the user's email (e.g., from a support ticket).
- `findAccountByAuthUserId` when you have a Serverpod auth user ID and want the linked account.



- `findUserByFirebaseUserId` when you have a Firebase UID and want the Serverpod user it is linked to.

```
final accountByEmail = await admin.findAccountByEmail(
  session,
  email: 'user@example.com',
);

final accountByAuthUser = await admin.findAccountByAuthUserId(
  session,
  authUserId: authUserId,
);

// findUserByFirebaseUserId is static, unlike the instance methods above.
final userId = await FirebaseIdpAdmin.findUserByFirebaseUserId(
  session,
  userIdentifier: 'firebase-uid',
);
```

Linking Firebase authentication

Link an existing Serverpod user to a Firebase account. This is useful when migrating users from another auth provider to Firebase, or when manually linking accounts in an admin tool.

First, obtain the account details from a Firebase ID token, then link them to an existing user:

```
// Fetch account details from a Firebase ID token
final accountDetails = await admin.fetchAccountDetails(
  session,
  idToken: firebaseIdToken,
);

// Link the account to an existing user
await admin.linkFirebaseAuthentication(
  session,
  authUserId: existingUserId,
  accountDetails: accountDetails,
);
```

Deleting accounts

Use `deleteFirebaseAccount` when you have a Firebase UID, or `deleteFirebaseAccountByAuthUserId` to remove every Firebase link attached to a single Serverpod user:

```
final deletedByUid = await admin.deleteFirebaseAccount(
  session,
  userIdentifier: 'firebase-uid',
);
```

```
);

final deletedByAuthUser = await admin.deleteFirebaseAccountByAuthUserId(
  session,
  authUserId: authUserId,
);
```

WARNING

Deleting a Firebase account only removes the link between Firebase authentication and the Serverpod user. The Serverpod user stays in your database, and the Firebase user stays in your Firebase project. You must delete those separately if that is what you want.

FirebaseIdpUtils

The `FirebaseIdpUtils` class provides a lower-level `authenticate` method for when you need to verify a Firebase ID token and find or create the associated Serverpod user in custom endpoint logic (outside the normal sign-in flow):

```
final utils = firebaseIdp.utils;

final authSuccess = await utils.authenticate(
  session,
  idToken: firebaseIdToken,
  transaction: transaction,
);
```